

QR Text Reader

Küçük, hızlı, **tamamen offline** bir Linux masaüstü aracı. Ekranda gördüğünüz herhangi bir QR kodunu seçip, içindeki **ham (raw) metni** olduğu gibi gösterir ve panoya (clipboard) kopyalar.

Ham metin her zaman değişmeden gösterilir/kopyalanabilir. Ek olarak, eğer QR bir **otpauth://** URI'siyse (2FA kurulum QR'ları), uygulama **sadece kolaylık için secret** ve **issuer/publisher** alanlarını ayrıca ayrıştırıp gösterir ve her biri için ayrı bir "Copy" butonu sunar. Bu bir TOTP kodu **üretmez** (authenticator değildir) — QR'da zaten açıkça bulunan bilgiyi sadece daha kolay kopyalanabilir hale getirir.

QR kodu → ham metin → clipboard
↳ (otpauth ise) secret → ayrı clipboard kopyası
↳ (otpauth ise) issuer → ayrı clipboard kopyası

Hedef platform: **Fedora Linux, GNOME masaüstü, Wayland.**

1. Nasıl çalışır?

"Scan QR" butonu veya pencere odaktaiken Ctrl+Shift+Q
↓
GNOME Shell'in native ekran-bölgesi-seçim arayüzü açılır (xdg-desktop-portal Screenshot portal, interactive mod)
↓
Seçilen alan RAM'e okunur, decode edilir, geçici dosya hemen silinir
↓
Ham metin pencerede gösterilir + panoya kopyalanır + bildirim gösterilir
↓
(otpauth ise) Secret ve Issuer alanları da ayrıca gösterilir, her biri kendi "Copy" butonuyla ayrı ayrı kopyalanabilir

Not: Artık ayrı bir "headless scan" (**--scan**) modu yok. Her tarama her zaman pencere içinde başlatılır ve sonuç her zaman pencerede

görünür — böylece ne kopyalandığını asla tahmin etmeniz gerekmez.

Bkz. [docs/architecture bölümü aşağıda](#) için detaylar.

2. Fedora üzerinde kurulum

2.1 Hızlı kurulum (önerilen)

Proje klasörünün içinden:

```
chmod +x install.sh # zip'ten çıkarıldıysa çalıştırma izni gerekebilir
./install.sh
```

Bu betik:

1. Gerekli **dnf** paketlerini kontrol eder, eksikse (izinizle) **sudo dnf install** ile kurar.
2. **~/local/share/qr-text-reader/venv** altında **--system-site-packages** ile bir sanal ortam oluşturur (GTK4/PyGObject sistem paketlerini görebilmesi için).
3. Uygulamayı bu sanal ortama kurar.
4. **~/local/bin/qr-text-reader** komutunu bağlar (yeni bir terminalde **qr-text-reader** çalışır).
5. **.desktop** dosyasını ve ikonu kurup GNOME uygulama menüsüne ekler.

Kurulumdan sonra **yeni bir terminal açın** veya **hash -r** çalıştırın, ardından:

```
qr-text-reader
```

GNOME Etkinlikler/Uygulama menüsünden de **"QR Text Reader"** olarak açabilirsiniz.

Kaldırmak için:

```
./uninstall.sh
```

2.2 Manuel kurulum (venv ile, adım adım)

install.sh'in ne yaptığını görmek veya elle kontrol etmek isterseniz:

Sistem bağımlılıkları (dnf)

```
sudo dnf install -y \
python3 python3-pip \
python3-gobject gtk4 libadwaita \
zbar \
xdg-desktop-portal xdg-desktop-portal-gnome
```

zbar → *pyzbar*'ın kullandığı native decode kütüphanesi.

xdg-desktop-portal-gnome → GNOME'un interaktif ekran görüntüsü UI'ı için gerekli, Fedora Workstation'da normalde zaten kurulu gelir.

Proje kurulumu

```
python3 -m venv --system-site-packages .venv
# --system-site-packages ÖNEMLİ: PyGObject/GTK4 binding'leri dnf ile kurulan
# sistem Python'una bağlı; venv içine ayrıca pip ile PyGObject kurmak
# genelde native GTK4 lib'lerini bulamaz.
```

```
source .venv/bin/activate
pip install .
```

Çalıştırma

```
qr-text-reader # venv aktifken
```

2.3 RPM ile paketleme (ileri seviye, opsiyonel)

packaging/qr-text-reader.spec dosyası sağlanmıştır. Sistem çapında,

dnf'in yönettiği bir kurulum istiyorsanız:

```
sudo dnf install -y rpm-build python3-devel python3dist(setuptools) python3dist(wheel)
```

```
VERSION=0.1.0
mkdir -p ~/rpmbuild/{SOURCES,SPECS}
tar --transform "s,^,qr-text-reader-$VERSION,/" \
  -czf ~/rpmbuild/SOURCES/qr-text-reader-$VERSION.tar.gz \
  $(git ls-files 2>/dev/null || ls) # proje kökünden çalıştırın
cp packaging/qr-text-reader.spec ~/rpmbuild/SPECS/
```

```
rpmbuild -ba ~/rpmbuild/SPECS/qr-text-reader.spec
```

```
sudo dnf install -y ~/rpmbuild/RPMS/noarch/qr-text-reader-$VERSION-1*.noarch.rpm
```

Bu yol, `.desktop` dosyasını ve ikonu doğrudan `/usr/share/...` altına

kurar, tüm kullanıcılar için kullanılabilir hale getirir ve `sudo dnf remove qr-text-reader` ile temiz biçimde kaldırılabilir.

3. Masaüstü entegrasyonu

`install.sh` kullandıysanız bu adım zaten yapıldı — atlayabilirsiniz.

Manuel kurulum yaptıysanız:

```
mkdir -p ~/.local/share/applications ~/.local/share/icons/hicolor/scalable/apps
sed "s|^Exec=.*|Exec=$(pwd)/.venv/bin/qr-text-reader|" \
  data/com.example.QRTextReader.desktop \
  > ~/.local/share/applications/com.example.QRTextReader.desktop
cp data/icons/com.example.QRTextReader.svg \
  ~/.local/share/icons/hicolor/scalable/apps/
update-desktop-database ~/.local/share/applications/ 2>/dev/null || true
gtk-update-icon-cache -f ~/.local/share/icons/hicolor 2>/dev/null || true
```

Uygulama artık GNOME uygulama menüsünde "**QR Text Reader**" olarak, kendi ikonuyla görünür.

Kısayol (Ctrl+Shift+Q)

Bu, pencere odaktaiken çalışan bir **uygulama-içi** kısayoldur (global bir sistem kısayolu değildir — Wayland'da uygulamaların keyfi global hotkey register etmesi güvenlik nedeniyle zaten kısıtlıdır). Kullanım: uygulamayı açın, pencereye odaklanın, `Ctrl+Shift+Q`'ya basın veya doğrudan **Scan QR** butonuna tıklayın.

4. Kullanım

- **Scan QR** butonuna basın veya `Ctrl+Shift+Q` kısayolunu kullanın.
- GNOME'un ekran seçim arayüzünde QR kodunu içeren alanı seçin.
- `Enter`/onay ile seçimi tamamlayın, `Esc` ile iptal edin.
- Sonuç pencerede gösterilir:

- o **Raw text** kutusu → QR'ın tam, değiştirilmemiş içeriği. **Copy full text** ile kopyalanır.
- o Eğer içerik bir **otpauth://** URI'siyse, ayrıca **Issuer** ve **Secret** alanları görünür — her biri kendi Copy butonuyla ayrı ayrı kopyalanabilir.
- o Birden fazla QR bulunursa → hangi sonucun kullanılacağını seçmeniz istenir.
- o QR bulunamazsa/okunamazsa → anlaşılır bir hata mesajı gösterilir.
- Başarılı taramadan sonra ham metin otomatik olarak panoya da kopyalanır; **Ctrl+V** ile istediğiniz yere yapıştırabilirsiniz.

5. Mimari Özeti

Katman	Modül	Sorumluluk
Screenshot	screenshot.py	org.freedesktop.portal.Screenshot (interactive) ile RAM'e görüntü alma, geçici dosyayı okur okumaz silme
Decode	decoder.py	pyzbar ile QR → ham string (transformasyon yok, sadece decode)
OTP alan ayrıştırma	otp_parse.py	otpauth:// URI'lerinden secret/issuer/account çıkarımı (yalnızca gösterim amaçlı, TOTP kodu üretmez)
Clipboard	clipboard.py	GTK4 Gdk.Clipboard , opsiyonel gecikmeli temizleme
Orkestrasyon	scanner.py	Yukarıdakileri birleştirir, tek sorumluluk noktası
UI	window.py , main.py	GTK4/libadwaita pencere, --scan CLI bayrağı, tekil instance
Güvenlik	security.py	Hassas veri içermeyen loglama, güvenli dosya silme
Ayarlar	config.py	Minimal JSON config (~/.config/qr-text-reader/config.json)

Neden bu QR decoder?

pyzbar → **zbar** C kütüphanesinin ince bir Python binding'i. Tamamen offline, Fedora repolarında (**zbar**) mevcut, aktif ve olgun bir proje, minimum dependency ağacı.

Neden bu screenshot yöntemi?

Wayland'da uygulamalar ekranı doğrudan yakalayamaz — bu **compositor**

tarafından kasıtlı olarak engellenir. **xdg-desktop-portal**'ın

Screenshot(interactive=true) çağırısı, GNOME Shell'in kendi native

alan-seçim arayüzünü açar. Biz kendi ekran-karartma/seçim UI'ımızı

yazmıyoruz — bunu yazmak Wayland güvenlik modelini bypass etmeye çalışmak

anlamına gelirdi.

6. Güvenlik Modeli

- × Network isteği yok (statik olarak `tests/test_security.py` ile doğrulanır: `socket`, `requests`, `urllib.request`, `http.client` vb. import edilemez).
- × Telemetry / analytics / crash reporting yok.
- × QR içeriği hiçbir zaman log'a yazılmaz (`security.py` sadece sabit olay kodları loglar; testle doğrulanır).
- × QR içeriği diske yazılmaz.
- × Screenshot kalıcı olarak saklanmaz; portal'ın oluşturduğu geçici dosya okunur okunmaz silinir.
- × QR içeriği hiçbir shell komutunda / `subprocess` / `eval` / `exec` içinde kullanılmaz (statik testle doğrulanır) — kötü niyetli bir QR (`$rm -rf ...`) gibi sadece bir **string** olarak ele alınır, asla çalıştırılmaz.
- ✓ Uygulama normal kullanıcı yetkisiyle çalışır, root istemez.
- ⚠ **Clipboard dürüstlüğü:** Wayland'da panoya yazılan veri, bazı masaüstü ortamlarında uygulama kapandığında kaybolabilir veya kompozitor'a bağlı olarak korunabilir. Bu davranış bizim kontrolümüzde değildir; uygulama bu konuda **yanlış bir güvenlik garantisi vermez.**

Güvenlik testlerini çalıştırma

```
source .venv/bin/activate
pip install -r requirements.txt # pytest + qrcode (sadece test için)
pytest tests/ -v
```

Testler şunları kapsar: raw text korunumu (URL, otpauth, Türkçe karakter, uzun metin), bozuk QR/görüntü, çoklu QR, panoya yazma, **loglarda QR**

içeriğinin asla görünmediğinin doğrulanması, ve kaynak kodda network/shell execution çağrısı olmadığının statik analizi.

7. Paketleme Durumu

- **Kurulu ve hazır:** `install.sh` (kullanıcı-yerel venv + PATH + desktop entry) ve `packaging/qr-text-reader.spec` (RPM) — ikisi de yukarıda §2'de anlatıldı.
 - **Flatpak:** Henüz yok. GTK4/libadwaita için doğal bir seçenek olsa da, `zbar`'ı ve portal izinlerini (`--talk-name=org.freedesktop.portal.Desktop`) manifest'e doğru tanımlamak ek iş gerektiriyor; bilinçli olarak kapsam dışı bırakıldı.
-

8. Kaldırma

`install.sh` ile kurduysanız:

```
./uninstall.sh
```

Manuel (venv) kurulum yaptıysanız:

```
rm -rf .venv
rm ~/.local/share/applications/com.example.QRTextReader.desktop
rm ~/.local/share/icons/hicolor/scalable/apps/com.example.QRTextReader.svg
rm -rf ~/.config/qr-text-reader
```

RPM ile kurduysanız:

```
sudo dnf remove qr-text-reader
```

9. Kısıtlar / Kapsam Dışı

- **TOTP kodu üretmez** — sadece QR'da zaten açıkça bulunan `secret/issuer` metnini gösterip kopyalatır; hiçbir hesaplama/algoritma çalıştırmaz.
- Şifre saklamaz, veritabanı tutmaz, hiçbir alanı diske yazmaz.
- Ayırıştırılan `secret/issuer` de dahil hiçbir şey loglanmaz veya ağa gönderilmez.
- Sürekli arka planda ekran taraması yapmaz — sadece kullanıcı tetiklediğinde çalışır.
- Ayrı bir headless/`--scan` CLI modu yoktur; her sonuç her zaman pencerede görünür.